Crypt: Technical Architecture Specification

LLM-Directed Text Adventure with Go Game Engine

March 2026 — v0.4

Contents

1 Overview **3**
- 1.1 Mechanics Reference . . . 3
- 1.2 Design Principles . . . 3

2 Play Modes **3**
- 2.1 Inference Tier Chain . . . 4
- 2.2 Mode Responsibilities . . . 4

3 System Architecture **5**
- 3.1 Component Map . . . 5
- 3.2 Topology . . . 6
- 3.3 Data Flow: Normal Mode . . . 6
- 3.4 Data Flow: Passthrough Mode . . . 7
- 3.5 Daemon Concurrency Model . . . 7
  - 3.5.1 Game-as-Goroutine . . . 7
  - 3.5.2 Command Types . . . 7
  - 3.5.3 Session/Game Separation . . . 7
  - 3.5.4 Session Resume . . . 7
  - 3.5.5 Graceful Shutdown . . . 8
  - 3.5.6 Panic Recovery . . . 8

4 The Three Engine Interfaces **8**
- 4.1 CommandInterpreter . . . 8
- 4.2 Narrator . . . 8
- 4.3 Renderer . . . 9

5 Game Engine Design **9**
- 5.1 Core Data Model . . . 9
- 5.2 Character . . . 9
- 5.3 Map . . . 10
- 5.4 Combat . . . 10

6 Data Formats **10**
- 6.1 Scenario Format (YAML) . . . 10
- 6.2 Save File Format (JSON) . . . 11

7 JSON-RPC Method Surface (Game Engine) **12**
- 7.1 Protocol Methods . . . 12
- 7.2 Session Management Tools . . . 12
- 7.3 Navigation . . . 12
- 7.4 Inventory . . . 13
- 7.5 Combat . . . 13

8 Lux Display Integration **13**
8.1 HUD Layout . . . 13
8.2 Update Strategy . . . 14
8.3 Lux Tools Used . . . 14

9 mcp-proxy Integration **14**
9.1 Problem . . . 14
9.2 Solution . . . 14
9.3 Daemon Scope Constraint . . . 14

10 Inference Integration **15**
10.1 Three Tiers . . . 15
10.2 API Format . . . 15
10.3 Interpreter Strategy . . . 16
10.4 Narrator Strategy . . . 16

11 Biff Extension Points **16**

12 Implementation Phases **17**

13 Open Questions **17**

14 Test and Verification Architecture **17**
14.1 Philosophy . . . 18
14.2 Testing Pyramid . . . 18

1 Overview

Crypt is a text adventure game for Claude Code. In its current form the entire engine — parser, rules, narrator, state machine — is a single LLM skill prompt. This specification describes the evolved architecture: a deterministic Go game engine that handles all state transitions, with the LLM acting as Dungeon Master (narrator, cartographer, encounter designer) rather than parser.

The central insight is **play mode as a first-class concept**. A play mode is a named combination of three swappable Go interfaces: `CommandInterpreter`, `Narrator`, and `Renderer`. All modes share the same engine, the same save files, and the same scenario format. Switching modes mid-adventure — save in **dm**, resume in **solo** — is explicitly supported.

Note on companion documents. Three binaries now exist: `cryptd` (server), `crypt` (client), and `crypt-admin` (scenario authoring). The authoritative reference for implemented game mechanics (combat formulas, damage reduction, leveling tables, point-buy stats, consumables) is `docs/gameplay.tex`. This architecture document describes the system-level design; some mechanics sections below reflect design-stage aspirations that differ from the current implementation.

1.1 Mechanics Reference

The game targets the mechanical vocabulary of *Wizardry I* (grid-based dungeon, turn-based combat, four character classes, dungeon floors with stairs) and the exploratory sensibility of *Zork I* (free-text examination, rich environmental prose, consequence of curiosity). Where the two conflict, Wizardry's determinism wins: rules are authoritative, the engine adjudicates.

1.2 Design Principles

P1. **The engine adjudicates; the LLM narrates.** No game outcome is left to probabilistic language model output. State transitions, combat resolution, and inventory rules are computed deterministically in Go. The LLM enriches the result with prose.

P2. **Interfaces are the seams.** `CommandInterpreter`, `Narrator`, and `Renderer` are the only coupling points between the engine and external systems. Each can be replaced independently.

P3. **Play mode is operational context, not architecture.** The same binary supports DM, solo, and headless play. No code paths are conditional on mode; mode selects implementations.

P4. **Daemon scope stays minimal.** The shared game engine daemon handles game logic and push routing only. No LLM calls, no orchestration, no business logic outside the game model.

P5. **Save files are cross-mode.** A save created in **dm** mode is loadable in **solo** or **headless**. The `play_mode` field in the save is advisory; `--mode` overrides it at resume time.

P6. **Party is designed in, not bolted on.** `GameState.Party` is always a slice (length 1 for single player). Combat, movement, and save tooling accept an optional `character_id`. Multi-player (Biff Phase 5) requires no engine redesign.

2 Play Modes

Three play modes are first-class citizens of the system. Each is a named triple of interface implementations. The engine always runs as `cryptd serve`; there is no embedded deployment.

Interface	**dm**	**solo**	**headless**
`CommandInterpreter`	`LLMInterpreter`	`SLMInterpreter`	`RulesInterpreter`
`Narrator`	`LLMNarrator`	`SLMNarrator`	`TemplateNarrator`
`Renderer`	`RPCRenderer` or `LuxRenderer`	`RPCRenderer` or `CLIRenderer`	`CLIRenderer`
Engine deployment	daemon	daemon	daemon
Invoked by	`crypt` client → `cryptd serve`	`crypt` client → `cryptd serve`	`cryptd serve -t`

2.1 Inference Tier Chain

The server auto-selects the best available inference tier at startup. Each tier provides both a `CommandInterpreter` and a `Narrator` implementation. The chain is:

1. **Large (Claude API)**: `LLMInterpreter` + `LLMNarrator` via OpenAI-compatible `/v1/chat/completions` endpoint at `api.anthropic.com`. Activated by `CRYPTD_API_KEY` env var or `--api-key` flag. A startup probe validates the key before committing.
2. **Medium/Small (SLM)**: `SLMInterpreter` + `SLMNarrator` via ollama or any OpenAI-compatible server. Auto-detected by probing default endpoints. Preferred models: `gemma3:1b`, `llama3.2:3b`.
3. **Tiny (Rules+Templates)**: `RulesInterpreter` + `TemplateNarrator`. Deterministic fallback when no inference server is available. Zero external dependencies.

Both LLM and SLM interpreters use a rules-first strategy: deterministic commands ("go north", "inventory") are handled by `RulesInterpreter` without an inference call. The model is only invoked when the rules interpreter returns `unknown` or when the action references items/enemies by ambiguous name.

2.2 Mode Responsibilities

Concern	**dm**	**solo**	**headless**
Room narration	LLM	SLM	Template
Free-text parsing	LLM	SLM	Rules/regex
Scenario generation	LLM interactive	Pre-made only	Pre-made only
Encounter lore	LLM	Pre-made only	Pre-made only
State transitions	Engine	Engine	Engine
Combat resolution	Engine	Engine	Engine
Fog-of-war, save/load	Engine	Engine	Engine
Rendering	Lux or RPC	RPC or CLI	CLI

3 System Architecture

3.1 Component Map

Component	Language / Layer	Responsibility
Game engine	Go (L1)	All state transitions; character, map, inventory, combat, save/load. Zero LLM dependencies.
`CommandInterpreter`	Go interface	Maps player free-text to a structured `EngineAction`.
`Narrator`	Go interface	Generates human-readable prose from engine event results.
`Renderer`	Go interface	Presents game state to the player (Lux HUD or ANSI CLI).
`cryptd` server	Go binary	JSON-RPC 2.0 daemon; exposes direct named methods (`session.init`, `game.new`, `game.play`, `game.move`, ...) over Unix socket or TCP. Game-as-goroutine architecture: each game is a goroutine that exclusively owns its engine and state. Session registry tracks connections; game registry tracks game goroutines. Session mode (normal: interpreter + narrator + RPCRenderer; passthrough: direct `game.*` dispatch) is selected per-session during `session.init`.
LLM inference	Go (`internal/inference`)	OpenAI-compatible HTTP client for `/v1/chat/completions`. Used by `LLMInterpreter`, `LLMNarrator`, `SLMInterpreter`, and `SLMNarrator`. Supports Claude API (with API key auth) and ollama (no auth).
`crypt` client	Go binary	Thin CLI client; connects to `cryptd serve`, sends text, displays game with readline and HP/MP bars. Auto-starts server.
`crypt-admin`	Go binary	Scenario authoring; graph-first generation, validation, SQLite working format, YAML directory export (DES-027).
`eval-balance`	Go binary	Monkey test harness; parallel game sessions with weighted-random actions, JSON aggregate reports for balance tuning.
`internal/monkeytest`	Go package	`MonkeyRenderer`, `SessionMetrics`, parallel runner. Plugs into the game loop via the Renderer interface.
`internal/scengen`	Go package	Graph types, topology sources, visitors, YAML exporter, SQLite store. Used by `crypt-admin`; SQLite not linked into server or client.
SKILL.md	Prompt (L4)	DM role instructions for Claude; narration, scenario generation, lore, examination responses.
Lux	Python / ImGui	Native display surface; receives element trees from `LuxRenderer` via MCP tools.
ollama	External service	Serves SLM for `SLMInterpreter` and `SLMNarrator`. Auto-detected in interactive `-t` mode; not required for CI.

3.2 Topology

The engine always runs as `cryptd serve`. There is no embedded topology. Two modes determine how clients interact with the server:

Normal mode (crypt client -¿ cryptd serve)

```
crypt (thin client)
    |
    | NDJSON over Unix socket or TCP
    v
cryptd serve
    |-- Session Registry (map[sessionID]*Session)
    |-- Game Registry (map[gameID]*Game)
    |
    |-- Connection 1 (goroutine)
    | `-- Session A -> Game X (goroutine)
    | |-- engine (exclusive)
    | |-- state (exclusive)
    | `-- commands channel
    |-- Connection 2 (goroutine)
    | `-- Session B -> Game Y (goroutine)
    |
    `-- Shared: interpreter, narrator (stateless)
```

Normal mode flow: client sends free text via `play` JSON-RPC method. The server interprets it (via the selected inference tier), executes the resulting action on the engine, narrates the outcome, and returns display-ready text. The game loop runs inside the game goroutine via `RunLoop`, driven by an `RPCRenderer` that bridges the network connection to the game loop's `Renderer` interface.

Passthrough mode (crypt mcp -¿ cryptd serve)

```
Claude Code session
    |
    | stdio MCP <-> crypt mcp bridge
    |
    | NDJSON over Unix socket or TCP
    v
cryptd serve
    |-- Session/Game registries (same as normal)
    |-- session.init { mode: "passthrough" }
    |-- game.* methods dispatched directly to engine
    `-- Structured JSON result returned (no interpretation/narration)
```

Passthrough mode: the client selects mode by sending `session.init` with `mode="passthrough"`. No interpreter or narrator is involved. Each `game.*` request (`game.move`, `game.look`, `game.attack`, ...) is routed through the game goroutine's command channel for serialized state access and returns a structured JSON result. The LLM (Claude) acts as its own interpreter and narrator. The daemon serves normal and passthrough sessions simultaneously on the same socket; mode is per-session, not per-server.

3.3 Data Flow: Normal Mode

In normal mode, the game loop runs inside the game goroutine. The `RPCRenderer` bridges the network connection to the loop.

1. **Connection.** Client connects and sends `session.init` with an optional `session_id` and optional `mode` (omitted or `""` selects normal mode). Server returns `session_id` and `has_game` (true if this session already has an active game from a previous connection). The response is a direct JSON result, not a `ToolResult` wrapper.

2. **Game start or resume.** If `has_game` is false, client calls `game.new` with scenario and character params. Server creates a game goroutine, sends the initial room, then enters the game loop via `Game.RunLoop()`. If `has_game` is true, the server enters `resumeGameLoop` immediately — no `game.new` needed. The current room is re-rendered.

3. **Play loop.** Client sends `game.play` requests with free text. Inside the game goroutine: `RPCRenderer.Events()` reads the text, the interpreter converts it to an `EngineAction`, the engine executes it, the narrator generates prose, and `RPCRenderer.Render()` writes a `PlayResponse` back to the client.
4. **Quit or death.** The game loop exits. A final `PlayResponse` with `quit` or `dead` set to true is sent. The game goroutine remains alive (state preserved for session resume).

The inference tier (LLM, SLM, or rules+templates) is transparent to this flow. Solo and DM modes differ only in which interpreter and narrator implementations are injected at startup.

3.4 Data Flow: Passthrough Mode

Passthrough mode has no game loop. Each `game.*` JSON-RPC request is dispatched to the game goroutine via `Game.Send()`. The game goroutine executes the method, returns structured JSON, and the handler writes it back as a direct JSON-RPC result. Claude Code (running as the `crypt mcp` stdio bridge) acts as its own interpreter and narrator.

3.5 Daemon Concurrency Model

3.5.1 Game-as-Goroutine

Each `Game` is a goroutine that exclusively owns its `*engine.Engine` and `*model.GameState`. No mutex protects game state — serialization is by construction via a buffered command channel (`chan Command`, capacity 1). This eliminates an entire class of concurrency bugs: the game goroutine is the only code that reads or writes engine state.

3.5.2 Command Types

Three command types flow through the channel:

Type	Purpose
`tool`	Method dispatch: execute a named `game.*` JSON-RPC method against the engine and return the result. Used by passthrough mode for every request and by normal mode for `game.new`.
`run_loop`	Normal mode game loop: the game goroutine runs `game.NewLoop().Run()` internally, blocking until the player quits, dies, or disconnects. The `RPCRenderer` reads/writes the network connection from within the goroutine.
`inspect`	Safe state access: runs a caller-provided function inside the game goroutine with exclusive access to engine and state. Used for test assertions and handler-level state queries.

3.5.3 Session/Game Separation

A `Session` is connection identity — it persists across reconnects. A `Game` is an engine + state goroutine. Sessions and games are stored in separate registries (`map[string]*Session` and `map[string]*Game`), protected by a shared `sync.RWMutex`.

Today the mapping is 1:1 (one session owns one game). The architecture supports N:1 for future multiplayer: multiple sessions can reference the same game ID, with the game goroutine serializing all their commands.

3.5.4 Session Resume

Client sends `initialize` with its previous `session_id`. The server looks up the session in the registry. If the session exists and has a `gameID`, the `InitializeResult` sets `has_game` to true. The handler then enters `resumeGameLoop`, which sends a `run_loop` command with `SkipInitialRender=false` — the current room is re-rendered so the player sees their position after reconnecting.

3.5.5 Graceful Shutdown

`s.cancel()` cancels the server context. All active connections are closed (unblocking any blocked scanners). `wg.Wait()` blocks until all connection handler goroutines exit. Game goroutines exit via context cancellation (the `select` on `ctx.Done()` in `Game.run()`).

3.5.6 Panic Recovery

Both game goroutines and connection handler goroutines recover from panics via `defer recover()`. A panic in one game or connection does not crash the daemon or affect other sessions.

4 The Three Engine Interfaces

4.1 CommandInterpreter

Maps a player's free-text string to a structured `EngineAction` that the engine can execute deterministically.

```go
type CommandInterpreter interface {
    Interpret(ctx context.Context, input string,
              state GameState) (EngineAction, error)
}
```

Implementation	Used in	Mechanism
`LLMInterpreter`	**dm** (Large tier)	Claude API via OpenAI-compatible `/v1/chat/completions`; rules-first strategy (exact verbs handled without LLM call; LLM invoked only for ambiguous input needing ID resolution).
`SLMInterpreter`	**solo** (Medium tier)	ollama or any OpenAI-compatible server; uses `internal/inference` client; same rules-first fallback.
`RulesInterpreter`	**headless** (Tiny tier)	Regex/keyword matching over known verb–noun pairs; no model required. Also serves as fallback for both LLM and SLM interpreters on inference failure.

4.2 Narrator

Generates human-readable prose from an `EngineEvent`: the result of an action after the engine has resolved it.

```go
type Narrator interface {
    Narrate(ctx context.Context, event EngineEvent,
            state GameState) (string, error)
}
```

Implementation	Used in	Mechanism
`LLMNarrator`	**dm** (Large tier)	Claude generates rich contextual prose for atmospheric events (room descriptions, combat starts, deaths, level-ups, item examinations). Tactical events (attack damage, inventory changes) use the template fallback. Selective enrichment: LLM for flavour, template for precision.
`SLMNarrator`	**solo** (Medium tier)	ollama generates 1–3 sentences of atmospheric prose from room seed + event type. Same selective enrichment strategy as LLM.
`TemplateNarrator`	**headless** (Tiny tier)	Go text/template: "You move north. You are in the {room.name}." Also serves as fallback for both LLM and SLM narrators on inference failure.

4.3 Renderer

Presents the current `GameState` to the player after each turn. Also receives button interaction events in DM/solo modes.

```
type Renderer interface {
    Render(ctx context.Context, state GameState,
           narration string) error
    Events() <-chan InputEvent // button presses, free-text
}
```

Implementation	Used in	Mechanism
`RPCRenderer`	Normal mode daemon	Bridges the game loop to the network connection in normal mode. `Render()` serializes a `PlayResponse` as NDJSON. `Events()` reads `play` requests from the client's scanner. Runs inside the game goroutine.
`LuxRenderer`	`-t --lux` mode	Calls Lux MCP tools (`show`, `update`, `recv`) to drive the full HUD. Currently used in `-t` testing mode.
`CLIRenderer`	`-t` mode, **headless**	ANSI terminal; ASCII map, text HP bar, stdin input loop.

5 Game Engine Design

5.1 Core Data Model

```
type GameState struct {
    ScenarioID string
    PlayMode string // advisory; --mode overrides on load
    Timestamp string
    Character Character
    Party []Character // len 1 for single-player
    Dungeon DungeonState
    Log []LogEntry
}

type Character struct {
    Name string
    Class CharClass // Fighter | Mage | Thief | Priest
    Level int
    HP, MaxHP int
    MP, MaxMP int
    Stats Stats // STR INT DEX CON WIS CHA
    XP, Gold int
    Inventory []Item
    Equipped Equipment
    Conditions []Condition
}

type DungeonState struct {
    CurrentRoom string
    Visited []string
    RoomState map[string]RoomState
}
```

5.2 Character

- **Classes**: Fighter, Mage, Thief, Priest. Class determines HP dice, MP pool, available spells, and XP thresholds.

- **Stats**: STR, INT, DEX, CON, WIS, CHA. Bonuses computed by standard table (8–18 range, D&D-adjacent).
- **Equipment slots**: weapon, armour, ring, amulet.
- **Leveling**: XP thresholds defined per class in scenario YAML. Stat gains on level-up are deterministic.

5.3 Map

- Grid-based dungeon in the Wizardry I style (20×20 floor). Rooms are nodes; corridors are edges.
- **Pre-generated**: the DM creates the room graph before play begins, either by writing scenario YAML directly or via the `/crypt:create` skill command. Procedural generation is explicitly not used — it produces maps that are inconsistent on backtracking.
- **Fog of war**: unvisited rooms are opaque on the Lux map canvas. Room content is not revealed until the player enters.
- **Room types**: corridor, chamber, stairway (up/down), treasure vault, boss chamber.
- **Door kinds**: open, locked (requires key item), hidden (requires search action), one-way, trapped.

5.4 Combat

Turn-based, deterministic. No probabilities are left to the LLM.

- Initiative roll (DEX modifier + 1d10) determines turn order.
- **Player actions per turn**: Attack, Cast Spell, Use Item, Flee (DEX check), Defend (+AC until next turn).
- **Enemy AI**: patterns encoded in scenario YAML. Supported patterns: `aggressive` (always attack), `cautious` (flee below 30% HP), `support` (buff allies before attacking), `scripted` (fixed action sequence).
- Damage formula: weapon dice + STR modifier vs. target AC.
- Conditions: poisoned, stunned, confused, blessed, hasted. Duration in turns; applied/cleared by spells and items.
- XP is awarded at `end_combat`; loot rolls are seeded from scenario YAML drop tables.

6 Data Formats

6.1 Scenario Format (YAML)

Scenarios are pre-authored YAML files stored in `scenarios/`. The engine parses them at startup; the LLM never sees the raw YAML during play.

scenarios/classic-fantasy-dungeon.yaml (excerpt)

```yaml
id: classic-fantasy-dungeon
title: "The Grimhold Dungeon"
description: "A classic dungeon crawl..."
version: "1.0"
xp_thresholds:
  fighter: [0, 2000, 4000, 8000, 16000]
  mage: [0, 2500, 5000, 10000, 20000]

rooms:
  - id: entrance
    name: "Dungeon Entrance"
    description: "Stone stairs descend into darkness."
```

```
    exits:
      south: goblin_lair
    items: []
    enemies: []

  - id: goblin_lair
    name: "Goblin Lair"
    description: "The reek of smoke and old blood."
    exits:
      north: entrance
      east: { room: treasure_vault, door: locked, key: rusty_key }
    enemies:
      - template: goblin
        count: 3
    loot_table:
      - item: rusty_key
        drop_chance: 0.8

enemy_templates:
  goblin:
    hp_dice: "2d6"
    ac: 12
    attack_dice: "1d4"
    xp: 30
    ai: aggressive
```

6.2 Save File Format (JSON)

Save files live at `.dungeon/saves/<slot>.json` (gitignored).

.dungeon/saves/1.json (excerpt)

```
{
  "version": 1,
  "scenario": "classic-fantasy-dungeon",
  "play_mode": "dm",
  "timestamp": "2026-03-10T18:00:00Z",
  "character": {
    "name": "Aldric", "class": "fighter", "level": 3,
    "hp": 45, "max_hp": 60, "mp": 0, "max_mp": 0,
    "stats": {"str":16,"dex":12,"con":14,"int":9,"wis":10,"cha":11},
    "xp": 1240, "gold": 48,
    "inventory": [],
    "equipped": {"weapon": "rusty_sword"},
    "conditions": []
  },
  "party": [],
  "dungeon": {
    "current_room": "goblin_lair",
    "visited_rooms": ["entrance", "corridor_a", "goblin_lair"],
    "room_state": {
      "goblin_lair": {"cleared": true, "items_taken": ["rusty_key"]},
      "treasure_vault": {"door_locked": false}
    }
  },
  "adventure_log": []
}
```

`party` is always present (empty for single-player). `play_mode` is advisory; `--mode` overrides on resume.

7 JSON-RPC Method Surface (Game Engine)

The engine exposes game actions as direct named methods on JSON-RPC 2.0. There is no MCP `tools/call` framing: each method has its own params shape and returns a direct JSON result object (or a standard JSON-RPC error).

7.1 Protocol Methods

Method	Params	Returns
`session.init`	`session_id?`, `mode?` (`""` or `"passthrough"`)	`protocolVer` `capabilitie` `has_game`
`session.quit`	—	`ok`
`game.new`	`scenario_id?`, `character_name?`, `character_class?`, `stats?`	Initial room `PlayRespons` structured ro
`game.list_scenarios`	—	`scenarios[]` `description`
`game.list_sessions`	—	`sessions[]`: metadata
`game.play`	`text`	`PlayRespons` `dead?`, `next_level_`
`game.context`	—	DM-facing s text
`game.move`	`direction`	Room transit
`game.look`	—	Current room
`game.attack`, `game.defend`, `game.flee`	(action-specific)	Combat resul
`game.examine`, `game.inventory`, `game.drop`, `game.equip`, `game.unequip`	(action-specific)	Item/invento

7.2 Session Management Tools

Tool	Arguments	Returns
`new_game`	`scenario_id`, `character_name`, `character_class`	Room description, exits, items, hero summary
`save_game`	`slot?`	Slot name and save file path
`load_game`	`slot?`	Room description, exits, items, hero summary

7.3 Navigation

Tool	Arguments	Returns
`move`	`direction`	Room, exits, items; combat if enemies present
`look`	—	Current room description, exits, items, combat state

7.4 Inventory

Tool	Arguments	Returns
`pick_up`	`item_id`	Item details, hero summary
`drop`	`item_id`	Item details, hero summary
`equip`	`item_id`	Item, slot, hero summary
`unequip`	`slot`	Item, slot, hero summary
`examine`	`item_id`	Item details
`inventory`	—	Items, equipped, weight, capacity

7.5 Combat

Tool	Arguments	Returns
`attack`	`target_id?`	Damage, killed, hero summary, enemy turns
`defend`	—	Defence status, hero summary, enemy turns
`flee`	—	Success (DEX check), hero summary
`cast_spell`	`spell_id`, `target_id?`	Spell effect, MP cost, hero summary

8 Lux Display Integration

Status: partially implemented. The `LuxRenderer` and Lux element tree builder exist and work in `cryptd serve -t --lux` mode. The full four-panel HUD described below is future work.

8.1 HUD Layout

The `LuxRenderer` drives a four-panel HUD. Navigation and combat buttons deliver input events directly to the engine — no LLM round-trip is required for button-driven actions.

Lux HUD layout (schematic)

```
+--------------------------------------------------+
| Menu: [Game] [Character] [Map] |
+----------------------------+---------------------+
| | HP [========--] 45 |
| DUNGEON MAP | MP [----------] 0 |
| (draw canvas, | Fighter Lv3 XP 1k |
| grid rooms, +---------------------+
| fog of war, | EXITS |
| visited/unvisited, | [N] |
| player dot, | [W] [E] |
| locked doors) | [S] |
| | [D stairs] |
+----------------------------+---------------------+
| NARRATION LOG | COMBAT (when active) |
| (markdown, scrolling) | Goblin [=====--] 8hp |
| | [Attack] [Use Item] |
| | [Defend] [Flee] |
| | Round 2 - Your turn |
+----------------------------+---------------------+
```

8.2 Update Strategy

Event	Lux operation
Room transition (new room entered)	`lux.show()` — replace full narration panel, update map fog
HP/MP change during combat	`lux.update()` — patch progress bar elements in-place
Narration text appended	`lux.update()` — append to narration log list
Combat begins/ends	`lux.show()` — activate or deactivate combat panel
Map fog reveal (new room)	`lux.update()` — patch map canvas element
Nav button press (player)	`lux.recv()` event → engine directly

8.3 Lux Tools Used

The `LuxRenderer` uses the following Lux MCP tools:

- `show(scene_id, elements, title?)` — initial HUD render and full scene transitions.
- `update(scene_id, patches)` — incremental HP, MP, fog, and log updates within a scene.
- `recv(timeout?)` — poll for button interaction events (navigation presses, combat actions).

The engine does not use `show_table`, `show_dashboard`, or `show_diagram`; those are data-analysis surfaces.

9 mcp-proxy Integration

Status: future / design-stage. With M10's concurrent session support and direct TCP/Unix socket connections, mcp-proxy is less critical than originally envisioned. Multiple `crypt` clients can connect directly to the daemon. The proxy pattern is still planned for Claude Code MCP multiplexing, where stdio constraints require a shim process.

9.1 Problem

Claude Code spawns one MCP server process per session via stdio. If the DM's Claude session owns the engine process via stdio, a second player connection is architecturally impossible: there is no shared process to connect to, and the DM cannot push events to players.

9.2 Solution

mcp-proxy is a near-zero-cost Go shim (<10 ms startup, <10 MB RSS) that each Claude session spawns via stdio. The shim connects to a pre-running shared daemon over a persistent bidirectional transport (WebSocket or NDJSON Unix socket) and relays MCP messages. At connect time, mcp-proxy resolves the topmost `claude` ancestor PID and injects it as a session identity header.

The daemon uses session identity to:

- Route DM-privileged methods only to the DM session.
- Push `tools/list_changed` notifications to the DM when a player takes an action (future; see cryptd-90e.3).
- Route DM narration output to all player sessions.

9.3 Daemon Scope Constraint

The beads project accumulated 70 000 lines of daemon code before being deleted. This daemon must not repeat that mistake. The game daemon handles:

- Game state (authoritative `GameState` for the session).
- JSON-RPC method dispatch (receive call → engine method → return result).

- Push routing (future: `tools/list_changed`, narration broadcast).

It does not handle: orchestration logic, plugin lifecycle, or anything outside the game model. Note: the daemon does now manage sessions (session registry, resume) and optionally runs inference (interpreter + narrator in normal mode). These are scoped additions, not the unbounded accretion that killed the beads daemon.

10 Inference Integration

All inference calls use the OpenAI-compatible `/v1/chat/completions` wire format via the `inference.Client` type in `internal/inference`. The same client code works for Claude API, ollama, llama.cpp, and any other compatible server.

10.1 Three Tiers

Tier	Backend	Configuration
Large	Claude API (`api.anthropic.com`)	Activated by `CRYPTD_API_KEY` env var or `--api-key` flag. Default model: `claude-sonnet-4-20250514` (configurable via `--model`). Uses `Authorization: Bearer <key>` header. A startup probe validates the key with a 5 s timeout before committing to this tier. On probe failure, falls through to Medium.
Medium/Small	ollama or any OpenAI-compatible server	Auto-detected by probing default endpoints (`localhost:11434`). Preferred models: `gemma3:1b`, `llama3.2:3b`. Configurable via `--model`.
Tiny	Rules + Templates (no inference)	Deterministic fallback when no inference server is available. `RulesInterpreter` + `TemplateNarrator`. Zero external dependencies.

10.2 API Format

Inference request (all tiers)

```
POST /v1/chat/completions
{
  "model": "gemma3:1b",
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."}
  ],
  "temperature": 0.0,
  "max_tokens": 150
}
```

Property	Value
API format	OpenAI-compatible `/v1/chat/completions`
Default SLM model	gemma3:1b (≈815 MB)
Default LLM model	claude-sonnet-4-20250514
Fallback	`RulesInterpreter` on any inference failure
Timeout (LLM)	15 s
Timeout (SLM)	5 s
Response format	JSON for interpreters; plain text for narrators

10.3 Interpreter Strategy

Both `LLMInterpreter` and `SLMInterpreter` use a rules-first strategy: the `RulesInterpreter` handles the input first. If it produces a definitive action without an ambiguous ID reference, the result is used directly (no inference call). The model is invoked only when rules return `unknown` or when the action needs ID resolution against game state context.

10.4 Narrator Strategy

Both `LLMNarrator` and `SLMNarrator` use selective enrichment: atmospheric events (room descriptions, combat starts, deaths, level-ups, item examinations) are sent to the model. Tactical events (attack results, inventory changes, movement confirmations) use the `TemplateNarrator` fallback for precision and speed.

11 Biff Extension Points

The engine is party-ready from day one. No redesign is required when Biff multi-player (Phase 5) is added.

- `GameState.Party` is always a `[]Character` slice; single-player sets `len(Party) == 1`.
- `move`, `attack`, `flee`, `defend` all accept an optional `character_id`; omitting it targets the primary character.
- Combat turn order already handles multiple actors.
- Save files carry a `party` array (empty until Biff).

When Biff is added:

- Each Biff participant controls one character in the party.
- `PostToolUse` hooks notify party members of state-change events.
- The DM narrates for the full party.
- Turn tokens are passed via Biff `/write` messages.

12 Implementation Phases

Phase	Deliverable	Status
M0 — Foundation	Project scaffold, Go module, CI, scenario parser, dice parser, save format, data contracts (`GameState`, `Character`, `EngineAction`, `EngineEvent`).	Complete
M1 — Data Contracts	All model types, interface definitions, scenario YAML schema, save/load with forward compatibility.	Complete
M2 — Thin E2E Slice	End-to-end architecture validation: engine core (move, look, pick_up), `RulesInterpreter`, `TemplateNarrator`, `CLIRenderer`, `cryptd serve -t` mode, e2e test framework.	Complete
M8 — Server Thin Slice	`cryptd serve` daemon with JSON-RPC 2.0 over Unix socket and TCP; `crypt` thin client; passthrough and normal modes; direct named `game.*` methods; daemonization; PID file management.	Complete
M9 — LLM Integration	Three-tier inference chain (Claude API → ollama SLM → rules+templates); `LLMInterpreter`, `LLMNarrator`, `SLMInterpreter`, `SLMNarrator`; auto-detection; `internal/inference` OpenAI-compatible client.	Complete
M10 — Concurrent Sessions	Game-as-goroutine; session registry; game registry; session resume via `session_id` in `session.init`; concurrent connection handling; `RPCRenderer`; panic recovery; graceful shutdown.	Complete
Future — DM Mode + Lux HUD	Full Lux HUD (map canvas, HP/MP bars, nav buttons); SKILL.md rewritten for DM role; `/crypt:create` — interactive scenario generation.	Planned
Future — Rich World	Shops, traps, cursed items; multiple bundled scenarios; DM-generated encounters with backstory and puzzles.	Planned
Future — Biff Multi-player	Party management (one character per Biff participant); N:1 session-to-game mapping; turn tokens via Biff `/write`.	Planned

13 Open Questions

Q1. **mcp-proxy dependency.** Still open but less urgent. With M10's direct TCP/Unix socket connections and concurrent session support, multiple `crypt` clients connect directly to the daemon. The proxy pattern is still needed for Claude Code's stdio MCP multiplexing but is no longer a blocker for multi-client play.

Q2. **SLM model selection.** *Resolved.* Default: `gemma3:1b`. Configurable via `--model` flag. Auto-detected by probing ollama endpoints.

Q3. **Go MCP SDK.** *Resolved.* Hand-rolled JSON-RPC 2.0 in `internal/daemon` and `internal/protocol`. Approximately 500 lines; no external dependency.

Q4. **Dice notation parser.** *Resolved.* Hand-rolled in `internal/dice`. Supports `NdS`, `NdS+M`, `NdS-M`.

Q5. **Permadeath default.** *Resolved.* Configurable per scenario in YAML. Hero death ends the game session (`dead` flag in `PlayResponse`).

14 Test and Verification Architecture

The authoritative testing reference is `docs/testing.md`. This section provides a brief summary.

14.1 Philosophy

The single most important property of the test suite is that it tells you immediately and unambiguously whether a change broke something. Three principles follow from this:

P1. **Determinism is the invariant.** The engine (Section 5) must always produce the same output for the same input. Every test that touches engine logic asserts a deterministic outcome. No probabilistic assertions, no "roughly correct."

P2. **Interface seams are injection points.** `CommandInterpreter`, `Narrator`, and `Renderer` (Section 4) are the only coupling points to external systems. All tests above unit level use test doubles at these seams. No real LLM, SLM, or Lux instance is ever required to run CI.

P3. **Headless mode is the CI workhorse. headless** uses `RulesInterpreter` + `TemplateNarrator` + `CLIRenderer` — all zero-dependency implementations. It runs complete game sessions programmatically and is the primary vehicle for integration and end-to-end tests.

14.2 Testing Pyramid

Layer	Trigger	Target	Coverage
Unit	Every commit	<5 s	Pure engine functions, parsers, `RulesInterpreter`, `TemplateNarrator`
Integration	Every commit	<30 s	Interface compositions, fake SLM, fake Lux, daemon routing, session resume
End-to-End	Every PR	<2 min	Real `cryptd` subprocess; MCP wire smoke; save round-trips
Acceptance	Release	<10 min	YAML game scripts through `cryptd serve -t`; full adventures

All four layers require zero external services. The full unit and integration pass runs in under 30 seconds. See `docs/testing.md` for the full test architecture: layer details, test double reference, fixture layout, CI configuration, and verification gates beyond tests (MCP schema contract, race detection, save format forward compatibility).